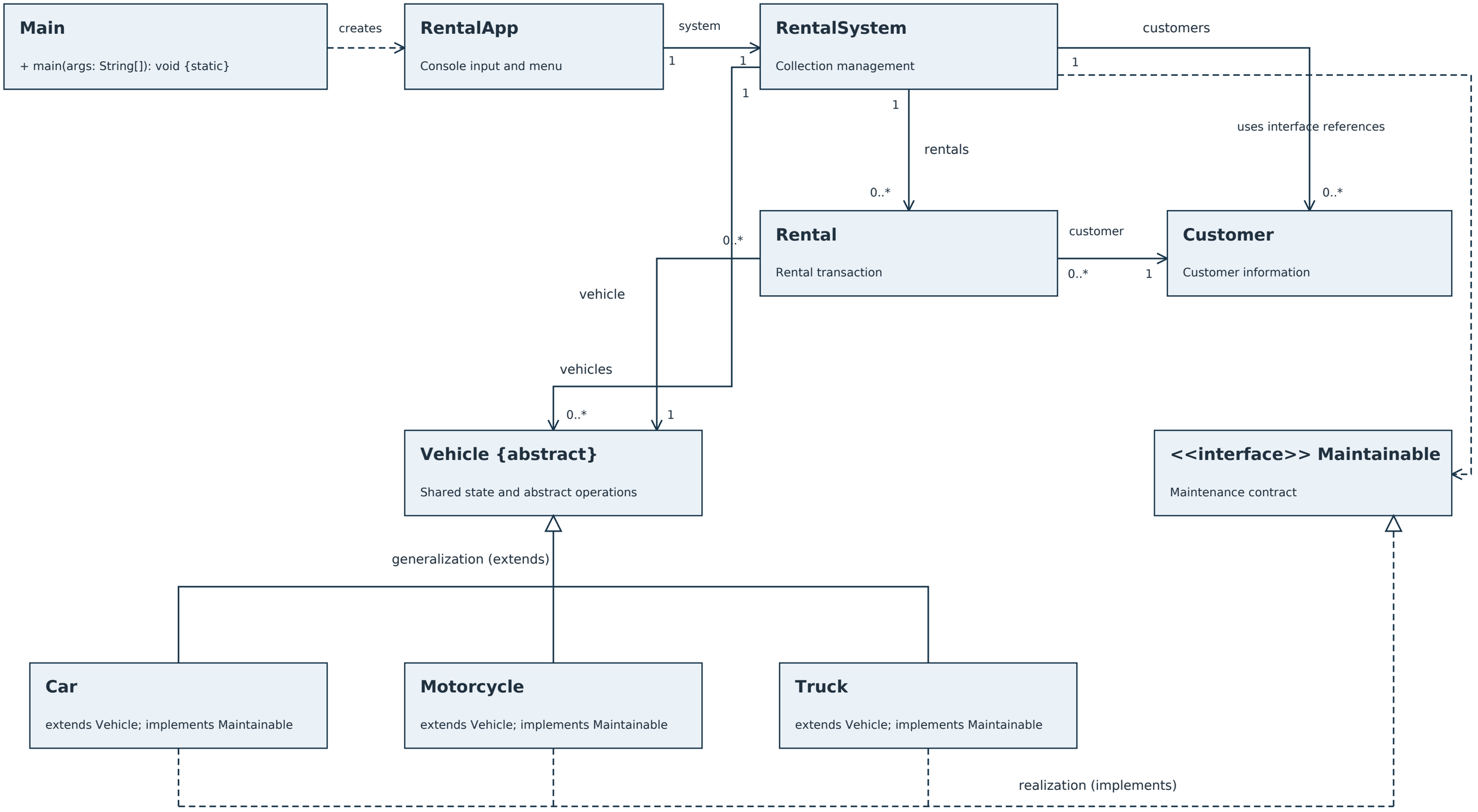
UML class diagram

Relationships and multiplicities. Pages 2-3 list declared fields and operations; inherited members are not repeated.



Solid arrow: navigable association. Hollow triangle: inheritance. Dashed hollow triangle: interface implementation.

Each rental has exactly one customer and one vehicle. A vehicle can have many historical rentals, but at most one active rental.

Vehicle classes and maintenance

Declared members | {abstract}: no implementation | {static}: shared member | {readOnly}: final constant

Vehicle {abstract}

- vehicleId: String
- brand: String
- model: String
- year: int
- dailyPrice: double
- available: boolean
- maintenanceRequired: boolean

+ Vehicle(vehicleId: String, brand: String, model: String, year: int, dailyPrice: double)
+ getVehicleId(): String
+ getBrand(): String
+ getModel(): String
+ getYear(): int
+ getDailyPrice(): double
+ setDailyPrice(dailyPrice: double): void
+ isAvailable(): boolean
+ isMaintenanceRequired(): boolean
+ flagForMaintenance(): void
+ rent(): void
+ returnVehicle(): void
completeMaintenance(): void
calculateBaseCost(days: int): double
displayBasicInfo(): void
+ calculateRentalCost(days: int, extraService: boolean): double {abstract}
+ displayInfo(): void {abstract}

<<interface>> Maintainable

(none)

+ performMaintenance(): void
+ needsMaintenance(): boolean

Car

- seats: int
- transmissionType: String
- INSURANCE_PER_DAY: double = 15 {static, readOnly}

+ Car(vehicleId: String, brand: String, model: String, year: int, dailyPrice: double, seats: int, transmissionType: String)
+ getSeats(): int
+ getTransmissionType(): String
+ calculateRentalCost(days: int, insurance: boolean): double
+ displayInfo(): void
+ needsMaintenance(): boolean
+ performMaintenance(): void

Motorcycle

- engineCapacity: int
- helmetIncluded: boolean
- HELMET_PER_DAY: double = 5 {static, readOnly}

+ Motorcycle(vehicleId: String, brand: String, model: String, year: int, dailyPrice: double, engineCapacity: int)
+ getEngineCapacity(): int
+ isHelmetIncluded(): boolean
+ setHelmetIncluded(helmetIncluded: boolean): void
+ calculateRentalCost(days: int, helmet: boolean): double
+ displayInfo(): void
+ needsMaintenance(): boolean
+ performMaintenance(): void

Truck

- maxLoadCapacity: double
- SERVICE_FEE: double = 50 {static, readOnly}

+ Truck(vehicleId: String, brand: String, model: String, year: int, dailyPrice: double, maxLoadCapacity: double)
+ getMaxLoadCapacity(): double
+ calculateRentalCost(days: int, extraService: boolean): double
+ displayInfo(): void
+ needsMaintenance(): boolean
+ performMaintenance(): void

Customers, rentals and application

Signatures match the Java code. Main.main(String[]) is shown on page 1. Constructor overloads are listed separately.

RentalSystem

- vehicles: ArrayList<Vehicle>
- customers: ArrayList<Customer>
- rentals: ArrayList<Rental>

+ RentalSystem()
+ addVehicle(vehicle: Vehicle): void
+ findVehicle(vehicleId: String): Vehicle
+ updateVehiclePrice(vehicleId: String, price: double): void
+ displayVehicles(): void
+ displayVehicles(availableOnly: boolean): void
+ displayRentedVehicles(): void
+ registerCustomer(customer: Customer): void
+ findCustomer(customerId: String): Customer
+ updateCustomerPhone(customerId: String, phone: String): void
+ updateCustomerEmail(customerId: String, email: String): void
+ displayCustomers(): void
+ rentVehicle(customerId: String, vehicleId: String, days: int, extraService: boolean): Rental
+ findRental(rentalId: int): Rental
+ returnVehicle(rentalId: int): void
+ displayRentals(): void
+ displayRentals(activeOnly: boolean): void
+ displayCustomerHistory(customerId: String): void
+ displayCustomerRentedVehicles(customerId: String): void
+ calculateTotalRevenue(): double
+ flagVehicleForMaintenance(vehicleId: String): void
+ displayVehiclesNeedingMaintenance(): void
+ performVehicleMaintenance(vehicleId: String): void

RentalApp

- system: RentalSystem
- input: Scanner

+ RentalApp()
+ run(): void
- displayMenu(): void
- readText(message: String): String
- readInt(message: String): int
- readDouble(message: String): double
- readYesNo(message: String): boolean
- addVehicle(): void
- searchVehicle(): void
- registerCustomer(): void
- rentVehicle(): void
- returnVehicle(): void
- displayCustomerHistory(): void
- flagMaintenance(): void
- performMaintenance(): void
- updateVehiclePrice(): void
- searchCustomer(): void
- updateCustomerPhone(): void
- updateCustomerEmail(): void
- displayCustomerRentedVehicles(): void

Customer

- customerId: String
- fullName: String
- phone: String
- email: String
- drivingLicenseNumber: String

+ Customer(customerId: String, fullName: String, phone: String, email: String, drivingLicenseNumber: String)
+ Customer(customerId: String, fullName: String, phone: String, drivingLicenseNumber: String)
+ getCustomerId(): String
+ getFullName(): String
+ getPhone(): String
+ getEmail(): String
+ getDrivingLicenseNumber(): String
+ setPhone(phone: String): void
+ setEmail(email: String): void
+ displayInfo(): void

Rental

- nextRentalId: int = 1 {static}
- rentalId: int
- customer: Customer
- vehicle: Vehicle
- days: int
- totalCost: double
- active: boolean

+ Rental(customer: Customer, vehicle: Vehicle, days: int, extraService: boolean)
+ getRentalId(): int
+ getCustomer(): Customer
+ getVehicle(): Vehicle
+ getDays(): int
+ getTotalCost(): double
+ isActive(): boolean
+ getStatus(): String
+ completeRental(): void
+ displayInfo(): void

Key constraints

Vehicle and customer IDs are unique.

Rental IDs are generated automatically.

A vehicle has at most one active rental.

Maintenance blocks renting.

An active rental blocks maintenance.

Revenue counts completed rentals only.